Jacobi Iteration on a 2D Grid: Serial, OpenMP, and Hybrid MPI+OpenMP Parallelisation

March 2026

Contents

1 The Numerical Problem **2**

2 Code Structure and Implementation **2**

2.1 Common baseline (`jacobi`) . . . 2

2.2 OpenMP version (`jacobi_just_omp`) . . . 2

2.3 MPI+OpenMP version (`jacobi_omp_mpi`) . . . 3

2.3.1 Domain decomposition . . . 3

2.3.2 Halo exchange . . . 3

2.3.3 Global residual . . . 3

2.3.4 Thread safety . . . 4

2.3.5 Timing . . . 4

3 Pipeline **4**

3.1 Expected scaling behaviour . . . 4

4 Performance Analysis **5**

5 Conclusions **5**

1 The Numerical Problem

The problem is to solve *steady-state Laplace equation*

$$\nabla^2 u(x,y) \;=\; \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \;=\; 0, \qquad (x,y) \in \Omega \subset \mathbb{R}^2, \tag{1}$$

using the uniform discretization of the domain $N_x \times N_y$ grid with grid spacing h in both directions. Including one layer of boundary (halo) cells, the computational array has size $(N_x+2)\times(N_y+2)$ and is stored in row-major (C) order as a flat 1D `double` array. The linear index of cell (i,j) is

$$k(i,j) = i\,(N_y+2) + j, \qquad 0 \le i \le N_x+1, \quad 0 \le j \le N_y+1. \tag{2}$$

using a scheme where at each iteration the interior values are updated via the five-point stencil

$$u_{i,j}^{(t+1)} \;=\; \frac{1}{4}\Big(u_{i-1,j}^{(t)} + u_{i+1,j}^{(t)} + u_{i,j-1}^{(t)} + u_{i,j+1}^{(t)}\Big), \qquad 1 \le i \le N_x,\; 1 \le j \le N_y. \tag{3}$$

The method requires two grids (`grid` and `grid_new`); after each sweep the new values are copied back to the active grid, and it is initialized via some boundary conditions. Iterations stop when the relative residual norm falls below a tolerance $\varepsilon = 10^{-4}$, or after a maximum of 5×10^5 iterations. The residual is computed before each sweep as

$$r^{(t)} \;=\; \sqrt{\sum_{i=1}^{N_x}\sum_{j=1}^{N_y}\Big(4\,u_{i,j}^{(t)} - u_{i-1,j}^{(t)} - u_{i+1,j}^{(t)} - u_{i,j-1}^{(t)} - u_{i,j+1}^{(t)}\Big)^2}, \tag{4}$$

and convergence is declared when $r^{(t)}/r^{(0)} < \varepsilon$.

2 Code Structure and Implementation

Three C99 executables are provided, sharing the same CLI `./jacobi_* <omp> <mpi>`. All functions are declared in a companion `.h` header so that the `.c` translation unit contains only `main`.

2.1 Common baseline (`jacobi`)

The header files `jacobi.h` provides four functions called for the pipeline and are defined with different parallelization techniques according to the usage case:

1. `fill_boundary_grid` — sets the four boundary rows/columns in both `grid` and `grid_new`.
2. `fill_grid_withouth_boundary` — zero-initialises all interior cells.
3. `update` — performs one Jacobi sweep (equation (3)).
4. `check_convergence` — evaluates the sum of squared residuals (equation (4)), without the square root, so that the caller can accumulate contributions from multiple sub-domains.

2.2 OpenMP version (`jacobi_just_omp`)

The header `jacobi_just_omp.h` parallelises the two compute-intensive kernels with OpenMP pragmas:

Listing 1: OpenMP loop parallelism in `update`.

```
#pragma omp parallel for schedule(static)
for (i = 1; i <= nx; i++) {
    for (j = 1; j <= ny; j++) {
        k = (ny+2)*i + j;
        grid_new[k] = 0.25 * (grid[k-1] + grid[k+1]
                              + grid[k-(ny+2)] + grid[k+(ny+2)]);
    }
}
```

Listing 2: `check_convergence`.

```
#pragma omp parallel for collapse(2) schedule(static) reduction(+:
   tmpnorm)
for (i = 1; i <= nx; i++) {
    for (j = 1; j <= ny; j++) { ... tmpnorm += diff*diff; }
}
```

`update` is partitioned statically since the grid is static and you do not have surprises on indices. In `check_convergence` use the collapse(2) to flatten the matrix The `reduction(+:tmpnorm)` clause lets each thread accumulate into a private copy; OpenMP merges the copies at the end of the parallel region, eliminating any data race on the accumulator. Timing is performed with `omp_get_wtime()`, called exclusively from the master thread before and after each parallel region to avoid concurrent writes to the timing variables.

2.3 MPI+OpenMP version (`jacobi_omp_mpi`)

2.3.1 Domain decomposition

The N_x rows are partitioned into contiguous 1D strips among P MPI ranks. Rank r owns

$$local_nx(r) = \left\lfloor \frac{N_x}{P} \right\rfloor + \mathbf{1}[r < N_x \bmod P], \tag{5}$$

so remainder rows are distributed among the lowest-rank processes, keeping load imbalance to at most one row. Each rank allocates $(local_nx + 2) \times (N_y + 2)$ doubles, where the extra two rows are *ghost (halo) rows* storing data received from neighbouring ranks.

2.3.2 Halo exchange

After the convergence check and before each `update`, a non-blocking halo exchange is performed:

Listing 3: Non-blocking halo exchange (excerpt).

```
MPI_Isend(&grid[1*(ny+2)],          ny+2, MPI_DOUBLE, rank-1, 0, ...);
MPI_Irecv(&grid[0*(ny+2)],          ny+2, MPI_DOUBLE, rank-1, 1, ...);
MPI_Isend(&grid[local_nx*(ny+2)],   ny+2, MPI_DOUBLE, rank+1, 1, ...);
MPI_Irecv(&grid[(local_nx+1)*(ny+2)], ny+2, MPI_DOUBLE, rank+1, 0, ...);
MPI_Waitall(req_cnt, reqs, MPI_STATUSES_IGNORE);
```

Using `MPI_Isend`/`MPI_Irecv` instead of blocking variants avoids deadlock regardless of process ordering and allows the MPI library to pipeline transfers.

2.3.3 Global residual

Each rank computes a *local* squared residual (the sum of squared differences over its own rows, without the square root). `MPI_Allreduce(MPI_SUM)` accumulates these across all ranks; the

global square root is then taken exactly once:

$$r_{\text{global}}^{(t)} = \sqrt{\sum_{p=0}^{P-1} r_{\text{local},\,p}^{(t)}}. \tag{6}$$

This matches the serial formula (4) exactly.

2.3.4 Thread safety

`MPI_Init_thread` is called with the `MPI_THREAD_FUNNELED` level, guaranteeing that only the master thread issues MPI calls. All `#pragma omp parallel for collapse(2)` and `reduction` pragmas are confined inside functions in `jacobi_omp_mpi.h` recalling exactly the functions in `jacobi_just_omp.h`; MPI calls in `main` remain strictly outside any parallel region.

2.3.5 Timing

Each rank measures its own elapsed times with `MPI_Wtime()`. Before writing the CSV, `MPI_Reduce(MPI_MAX, root=0)` collects the *bottleneck* (slowest rank) time for each function. Only rank 0 opens and writes the output file, preventing concurrent file access.

3 Pipeline

All three targets are declared in a single `CMakeLists.txt`, The global compiler flag `-O3` applies to all targets. The Python driver `main.py` automates the full benchmark pipeline:

1. **CMake build** — configures and compiles all targets in a `build/` subdirectory inside the scripts folder.
2. **Parameter sweep** — generates one SLURM batch script per combination of:
   - grid sizes: 100, 150, 200, 250, 300, 350, 400;
   - OMP threads per rank (T): $1, 2, 4, 8$;
   - MPI ranks (P): $1, 2, 4, 8$ (only for `omp_mpi`).

   The total job count is $7 \cdot 16 = 112$ jobs. `analysis.py` to produce all figures described in Section 4.

Every executable appends one row to a shared CSV file whose path is read from the environment variable (default:

`total_time` spans from grid allocation to `free()`.

`update_time` and `check_convergence_time` are *cumulative* across all iterations.

For the MPI variant, all times are the per-function maximum over ranks (`MPI_Reduce(MPI_MAX)`), reflecting the true wall-clock bottleneck.

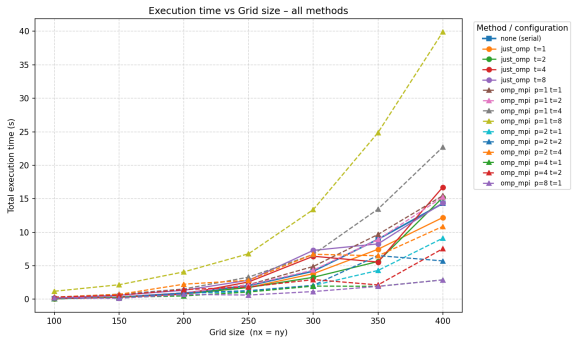
3.1 Expected scaling behaviour

Serial vs. OpenMP. For T threads Amdahl's law predicts a speed-up upper bound of $S(T) \leq 1/(f_s + (1 - f_s)/T)$, where f_s is the serial fraction (boundary setup, `memcpy`, loop overhead). For large grids $f_s \ll 1$ and near-linear scaling is expected up to the point where the problem fits in the last-level cache. Beyond that, memory bandwidth becomes the bottleneck and scaling saturates.

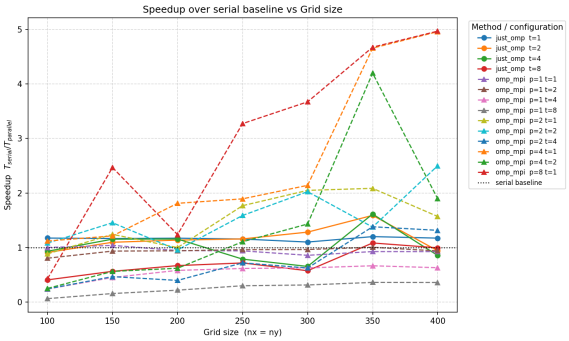
MPI strong scaling. With P ranks each processing N_x/P rows, the computation scales as $\mathcal{O}(N_x N_y/P)$, while the halo-exchange cost per iteration is $\mathcal{O}(N_y)$ (one row per neighbour). For fixed $N_x N_y$, increasing P reduces computation but keeps communication constant, so an efficiency drop is expected as $P \to N_x$.

Hybrid MPI+OpenMP. The hybrid configuration is expected to outperform pure MPI for a fixed total process count $P \cdot T$, because replacing T MPI ranks within a node with T OMP threads eliminates intra-node message passing. In practice the optimal split depends on memory-bandwidth saturation and network latency; the plots in this section expose these trade-offs empirically.

4 Performance Analysis



(a) Comparison time execution between all different methods tried computed via wall functions. We see that MPI works better with increasing performance with the number of cores used. OPENMP in MPI adds too much overhead in the communications compared to the size of the grid tested resulting in worse performance. OPENMP has some weird behavior around size grid = 350.



(b) Same as above just showing the ratio between serial and parallel time. We want to be above the horizontal dashed line for improvement. It is more easilu seen that OPENMP with 1 core introduces an overhead with respect to the serial that may account for the worsening of the MPI + OPENMP computations.

Figure 1

5 Conclusions

The key findings anticipated from the experimental plots are:

- **Performance of Pure MPI vs. Hybrid Configurations:** Execution using pure MPI (1 OpenMP thread) demonstrated the highest efficiency at smaller grid sizes. This is attributed to the fact that for low computational loads, the overhead associated with thread management—specifically the dispatching of computation and the `atomic` operations required for the error norm update—outweighs the parallel gain. In this regime, the synchronization cost is non-negligible compared to the grain size of the task.
- **OpenMP Scaling and Memory Anomalies:** OpenMP-only execution shows consistent scaling; however, a localized performance degradation (a "jump" in execution time) was observed at a grid size of 350. This non-linear behavior is unexpected and likely suggests a hardware-specific bottleneck, such as the memory footprint exceeding the L3 cache capacity or an increase in TLB (Translation Lookaside Buffer) misses.